RAG Assignment: Baseline vs. Re-Ranking

<Name> · <Roll number>

October 2026

Setup

Document. Apple Inc.'s Form 10-K for fiscal 2024: 121 pages, about 64,000 words. I chose it because it mixes very different kinds of text. Pages 1–60 are the annual report proper — financial statements, segment results, risk factors. Pages 61–121 are filed exhibits: debt indentures, employee stock plans, an insider trading policy and a compensation recovery (clawback) policy. Words like *hedge*, *pledge* and *compensation* mean different things in different parts of the document, which is exactly what a bi-encoder struggles with.

Chunking. 547 chunks of at most 200 tokens with a 40-token overlap, packed sentence by sentence. I sized chunks in tokens rather than words after finding that 150-word chunks left 6.9% of them longer than the bi-encoder's 256-token limit. Those chunks would have been silently truncated for the bi-encoder but not for the cross-encoder (limit 512), biasing the comparison in the re-ranker's favour. After the change, no chunk exceeds 229 tokens.

	Pipeline 1 — Baseline	**Pipeline 2 — Re-ranking**
Stage 1	`all-MiniLM-L6-v2`, Top 10 by cosine (FAISS)	same Top 10
Stage 2	—	`ms-marco-MiniLM-L-6-v2` scores all 10 pairs
Context	Top 3 by cosine	Top 3 by cross-encoder score
LLM	`qwen2.5:3b` (Ollama), temperature 0	identical model and prompt

The prompt tells the model to answer only from the context and to say so if the answer is missing. Since the candidate pool, prompt and model are identical, any difference between Answer 1 and Answer 2 comes from which three chunks were chosen. Everything ran on CPU.

1 Architecture comparison

A **bi-encoder** embeds the query and each chunk *separately* into fixed vectors and scores them by cosine similarity. Chunk vectors are computed once, offline, so a search costs one query embedding plus a fast vector lookup. The weakness is that each chunk is compressed into a single vector before the query is known. Everything the chunk says gets averaged together, and the model cannot weigh one word of the chunk against one word of the query. *"Employees may not hedge Apple stock"* and *"Apple hedges foreign currency exposure"* end up close in vector space because both are about hedging.

A **cross-encoder** concatenates the query and the chunk into one input and runs them through the transformer *together*, so self-attention connects every query token with every chunk token. It can notice that the query asks about employees and the chunk is about the company, or that a "not" changes the meaning. It outputs a relevance score rather than an embedding, so nothing can be precomputed: it must run once per (query, chunk) pair at query time. That makes it more accurate and much more expensive, which is why it only re-ranks the bi-encoder's ten candidates.

2 Qualitative and rank-shift analysis

Each query has a gold chunk — the one that actually contains the answer, found by reading the document. Table 1 shows where it ranked in each pipeline.

Where re-ranking fixed the answer (Q1–Q5)

In all five rescue queries the gold chunk sat at rank 5–7, inside the Top 10 but outside the Top 3, so the baseline never showed it to the LLM. Answer 1 was correct in **0 of 5** cases and Answer 2 in **4 of 5**.

Omitted answer (Q1, Q3, Q4). The baseline's context did not contain the answer, and the model correctly refused. For Q4 the baseline's Top 3 were a line from the exhibit index, the closing lines of the

Table 1: Rank of the gold chunk. Only the Top 3 reach the LLM.

	Query (category)	Baseline	Re-ranked	Outcome
Q1	Who serves as the Insider Trading Compliance Officer? *(buried)*	#7	#1	rescued
Q2	Is iPhone's share of net sales the same in every region? *(exception)*	#5	#1	rescued
Q3	Are puts and calls on Apple stock restricted for insiders? *(keyword)*	#7	#1	rescued
Q4	Which employees are not allowed to pledge Apple stock? *(negation)*	#7	#2	rescued
Q5	How is incentive-based compensation defined for clawback? *(keyword)*	#5	#1	rescued
Q6	Can executive officers pledge Apple stock as collateral? *(control)*	#1	#1	both fine
Q7	Why did sales in China fall in 2024? *(control)*	#1	#1	both fine
Q8	How many full-time equivalent employees did Apple have?	>10	>10	neither
Q9	Can shares be pledged?	>10	>10	neither
Q10	When is Apple excused from clawing back executive pay?	#7	#9	neither

clawback policy and a description of the employee stock plan — chunks that match "employees" and "executive officers" but say nothing about pledging. The re-ranker promoted the insider trading policy from #7 to #2:

Answer 1: "The context does not contain this information."

Answer 2: "Members of Apple's Board of Directors and executive officers are prohibited from pledging Apple stock as collateral."

Q4 is also the negation case. The question asks who is *not* allowed to do something; the cross-encoder read the query and the prohibition together and ranked the matching rule second, which the bi-encoder had placed seventh. Q3 is a distractor case of a different kind. The baseline's Top 3 were other sections of the same insider trading policy, on material non-public information and on trading plans. They share the query's vocabulary ("insiders", "Apple stock", "restricted") but none mentions puts, calls or derivatives. "Puts" appears in exactly one chunk of the whole document, in a list of prohibited instruments, and the cross-encoder found it.

Misstated rule (Q5). This was the most useful case, because the baseline answer *looked* right. It is also a keyword trap: the baseline's #2 and #3 were "Recoupment" clauses from employee stock award agreements, which overlap heavily with "clawback" and "compensation" but are not the clawback policy at all. With the gold chunk at #5, the baseline answered from its #1, a neighbouring chunk of the policy that describes which awards are in scope and when compensation counts as "received":

Answer 1: "Incentive-Based Compensation is defined as awards granted to a Covered Executive that are outstanding and unvested as of October 2, 2023, and is 'Received' in the fiscal period that the relevant financial reporting measure is attained…"

Answer 2: "Incentive-Based Compensation for clawback purposes includes any compensation granted, earned, or vested based in whole or in part upon the attainment of one or more financial reporting measures…"

Answer 1 is fluent and uses the right vocabulary, but it presents a scope rule as the definition. An automated check for the phrase "financial reporting measure" marked it correct, so every answer in the notebook was also labelled by hand.

Where re-ranking did not help (Q2, Q8–Q10)

Retrieval fixed, generation still failed (Q2). The re-ranker moved the right chunk to #1, but both answers said the context did not contain the information. The chunk states that the revenue mix was "generally consistent for each reportable segment…except in Greater China, where iPhone revenue represented a moderately higher proportion," wedged between sentences about deferred revenue. The 3B model did not connect that to the question. Re-ranking is necessary here, but not sufficient.

Recall ceiling (Q8). The gold chunk is not in the Top 10 even when the query uses the document's exact phrase, "full-time equivalent employees." The figure (164,000) is one sentence inside a chunk mostly about compensation and benefits, so the chunk's averaged vector is about compensation. The cross-encoder can only reorder the ten candidates it is given; it cannot recover a chunk the bi-encoder never retrieved.

Re-ranking made the answer worse (Q9). The document contains four near-identical restricted stock unit agreements, each saying unvested shares "shall not be sold, assigned, transferred, *pledged*…". These crowded the real answer out of the Top 10, and the cross-encoder promoted two of them into the Top 3 because they match "pledged" so closely. The baseline had refused; the re-ranked answer contradicts itself:

Answer 2: "Yes, shares can be pledged as per the restrictions on transfer mentioned in the context. Shares of Common Stock are not allowed to be sold, assigned, transferred, pledged…"

This is negation failing in the LLM, not the retriever: it saw "pledged" and answered yes.

Hallucination in both pipelines (Q10). The gold chunk says recovery can be excused if it would be "excessively burdensome," would cost more than it recovers, or would violate foreign law. It ranked #7 in the baseline and fell to #9 after re-ranking. Without it, both answers stated the opposite with confidence:

> *Answer 1: "Apple is not excused from clawing back executive pay under this policy… The policy does not provide a specific exemption or exception for when Apple is excused from such actions."*

This is the worst outcome in the set: a fluent, specific answer that is backwards. It also shows that a confident "no exception exists" from a RAG system can simply mean the exception was never retrieved.

3 Latency profiling

Each stage was timed over 20 repeats for each of the 10 queries (200 runs) after a warm-up pass. The LLM was timed separately on the five rescue queries. Ollama caches processed prompts, including from earlier runs, so repeated prompts looked about ten times faster than they really are; each timing call was given a unique prefix so that it had to be processed from scratch.

Stage (median, ms)	Pipeline 1	Pipeline 2
Query embedding (bi-encoder)	12.1	12.1
FAISS search over 547 vectors	0.4	0.4
Cross-encoder, 10 pairs	—	397.4
Retrieval total	**12.5**	**409.0**
LLM: reading the prompt	20,864	22,107
LLM: generating the answer	727	3,493
LLM total, incl. request overhead	21,642	29,149
End to end (retrieval + LLM)	**21,655**	**29,558**

Re-ranking makes retrieval about **33 times slower** (12.5 ms to 409 ms), but that is only **1.3%** of the end-to-end time, because a CPU-only LLM spends about 21 seconds just reading a three-chunk prompt. The larger end-to-end gap comes from answer length, not retrieval: Pipeline 2 produced real answers (median 38 tokens) where Pipeline 1 mostly said the context lacked the information (9 tokens). With a GPU-served or API model, generation would take a second or two and the 400 ms re-ranking cost would become a much larger share.

4 Scaling: why not cross-encode the whole collection?

Because the cross-encoder's cost grows with the number of chunks, and none of it can be done ahead of time. Every chunk must be read together with the query at query time. The bi-encoder does the expensive work once, offline; at query time it embeds one short query and compares vectors, which is a dot product per chunk.

I measured both on this document. Cross-encoding all 547 chunks for one query took **22.7 seconds** (41.5 ms per pair), against 0.36 s for the ten candidates. One cross-encoder pair costs as much as about 63,000 vector comparisons.

Chunks in collection	Cross-encode everything	Two-stage (flat index)
547	22.7 s	0.37 s
10,000	6.9 min	0.38 s
100,000	1.2 h	0.44 s
1,000,000	11.5 h	1.03 s

These are linear extrapolations from the measured per-pair and per-vector costs. The two-stage column uses an exact flat index, which still grows linearly but very slowly; approximate indexes such as HNSW grow sub-linearly. Even for this single document, a 22-second wait per query is too slow for interactive use, and at a million chunks it would take half a day. The two-stage design uses the cheap model to narrow the search and the expensive model only where precision matters.

Takeaways

- Re-ranking helped most when the right chunk was already in the Top 10 but buried by chunks that shared its vocabulary. It took the right answers from 0/5 to 4/5 on those queries.
- It cannot fix recall. If the bi-encoder misses the chunk (Q8, Q9), the cross-encoder never sees it, so the retrieval depth of the first stage matters as much as the re-ranker.
- Lexical decoys can fool the re-ranker too (Q9), and a missing exception produced the most dangerous answer in the set (Q10).
- On CPU the re-ranking cost (about 0.4 s) is small next to LLM generation, so the accuracy gain is clearly worth it here.

Code, outputs and data: `rag_baseline_vs_rerank.ipynb` in the repository. Temperature-0 generation was not word-for-word identical across three full runs (14 of 20 answers identical), but every answer's label held in all three.